

EE-170L Computer Programming Lab

Laboratory Report — No. 06

The for Loop in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	06
Topic	The for Loop, Nested for Loop
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

By the end of this lab session, students will be able to:

- Understand the syntax and structure of the for loop in C++.
- Write programs using the for loop to repeat actions a fixed number of times.
- Control loop execution with initialization, condition, and update expressions.
- Differentiate between for, while, and do-while loops and select the appropriate one.
- Implement nested for loops for multi-dimensional iteration tasks.
- Apply nested for loops to generate multiplication tables and structured patterns.

2. Theoretical Background

2.1 The for Loop

The for loop is a repetition control structure that allows efficient looping when the number of iterations is known in advance. It is used when a value needs to run from a starting point to an ending point with a constant increment or decrement.

Key characteristics of the for loop:

- The for loop is ideal when the number of repetitions is determined before the loop begins.
- All three loop-control elements (init, condition, update) are grouped in the loop header.
- The loop expression must evaluate to an integer or boolean condition.
- Any of the three parts can be omitted, but both semicolons inside the parentheses must remain.

Syntax:

```
for (initialization; condition; update variable)
{
    statements
}
```

2.2 Flow Control in a for Loop

The following steps describe precisely how a for loop executes at runtime:

- **Initialization:** The loop variable is declared and set once, before the first iteration.
- **Condition Check:** Before each iteration the condition is evaluated. If true, the body runs; if false, the loop terminates.
- **Loop Body:** The statements inside the braces execute for the current iteration.
- **Update Expression:** After the body finishes, the update expression runs (e.g., `i++`), then the condition is re-evaluated.
- **Termination:** Once the condition becomes false, control jumps to the statement immediately after the closing brace.

2.3 The Nested for Loop

A nested for loop is a for loop placed inside the body of another for loop. For every single iteration of the outer loop, the inner loop completes all its iterations. This pattern is used for two-dimensional tasks such as generating tables, processing grids, and printing patterns.

Structure:

```
for (int i = 1; i <= outer_limit; i++)
{
    for (int j = 1; j <= inner_limit; j++) // nested loop
    {
        // inner loop body
    }
}
```

3. Example Programs

3.1 Print Values from 5 to 1 Using for Loop

The following program demonstrates a decrementing for loop that counts down from 5 to 1 and prints each value:

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 5; i > 0; i--)
    {
        cout << i << endl;
    }
    cout << "Outside the loop now";
    return 0;
}
```

Output:

```
5
4
3
2
1
Outside the loop now
```

3.2 Prime Number Check Using for Loop

This program checks whether a user-entered number is prime by testing all divisors from 2 up to `num-1` using a for loop and a flag variable:

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int num;
    cout << "Enter a number please \n";
    cin >> num;
    int flag = 0;
    for (int i = 2; i < num; i++)
    {
        if (num % i == 0)
        {
            flag = 1;
            break;
        }
    }
    if (flag == 1)
        cout << num << " is not a prime number \n";
    else
        cout << num << " is a prime number \n";
    return 0;
}
```

3.3 Descending Pattern Using Nested for Loop

This program uses nested for loops to print a descending number pattern — the outer loop controls the row count, the inner loop prints numbers for that row:

```
#include <iostream>
using namespace std;
int main()
{
    for (int num = 7; num > 0; num--)
    {
        for (int j = 1; j <= num; j++) // nested loop
        {
            cout << j << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

Output:

```
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

4. Lab Tasks & Solutions

Task 1 — Display Numbers 1 to 10 Using for Loop

Write a C++ program that uses a for loop to display numbers from 1 to 10, with each number printed on a separate line.

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 1; i <= 10; i++)
    {
        cout << i << endl;
    }
}
```

```

    }
    return 0;
}

```

How it works:

- The loop variable *i* starts at 1 and increments by 1 after each iteration.
- The condition *i* <= 10 keeps the loop running until *i* becomes 11.
- Each iteration prints the current value of *i* followed by a newline using `endl`.
- The loop terminates automatically once the condition evaluates to false.
- Total output: 10 lines, one number per line from 1 through 10.

Task 2 — Multiplication Table Using Nested for Loop

Write a C++ program that generates a formatted multiplication table from 1 to 10 using nested for loops, with each product aligned in columns.

```

#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    cout << "Multiplication Table (1 to 10)" << endl;
    cout << "===== " << endl;
    for (int i = 1; i <= 10; i++)
    {
        for (int j = 1; j <= 10; j++)
        {
            cout << setw(5) << i * j;
        }
        cout << endl;
    }
    return 0;
}

```

How it works:

- The outer loop iterates rows (*i* from 1 to 10); the inner loop iterates columns (*j* from 1 to 10).
- *i* * *j* computes the product for each cell of the table.
- `setw(5)` from `<iomanip>` reserves 5 characters per value for neat column alignment.
- `endl` after the inner loop advances to a new line after each complete row.
- The result is a 10×10 grid showing all products from 1×1 = 1 to 10×10 = 100.

Task 3 — Sum of Numbers from 1 to N Using for Loop

Write a C++ program that prompts the user for a positive integer *n* and uses a for loop to calculate and display the sum of all integers from 1 to *n*.

```

#include <iostream>
using namespace std;
int main()
{
    int n, sum = 0;
    cout << "Enter a positive integer: ";
    cin >> n;
    for (int i = 1; i <= n; i++)
    {
        sum += i;
    }
    cout << "Sum from 1 to " << n << " = " << sum << endl;
    return 0;
}

```

```
}
```

How it works:

- The accumulator variable `sum` is initialized to 0 before the loop begins.
- The for loop runs from $i = 1$ up to and including n .
- `sum += i` adds the current loop value to the running total on every iteration.
- After the loop, the final accumulated sum is printed with a descriptive message.
- For example, if the user enters 5: Sum from 1 to 5 = 15 (since $1+2+3+4+5 = 15$).

Task 4 — Triangle Pattern Using Nested for Loop

Write a C++ program that uses nested for loops to print a right-triangle pattern of asterisks. The number of rows is entered by the user, and each row contains an increasing number of asterisks.

```
#include <iostream>
using namespace std;
int main()
{
    int rows;
    cout << "Enter number of rows: ";
    cin >> rows;
    for (int i = 1; i <= rows; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

How it works:

- The outer loop controls the current row number i , running from 1 to `rows`.
- The inner loop runs from $j = 1$ to $j = i$, so row i contains exactly i asterisks.
- Row 1 prints 1 asterisk, row 2 prints 2, and so on — forming a left-aligned triangle.
- `endl` after the inner loop advances to the next row.

Sample Output (rows = 5):

```
*
* *
* * *
* * * *
* * * * *
```

5. for Loop vs while Loop — Key Differences

Both `for` and `while` are iteration control structures in C++. The table below summarizes the key differences to help determine which to use:

Feature	for Loop	while Loop
Best use case	Known number of iterations	Unknown iterations, condition-based

Initialization	Inside the loop header	Before the loop body
Update location	Inside the loop header	Inside the loop body
Readability	Compact and self-contained	More flexible but more verbose
Common usage	Counting, tables, patterns	Input validation, event loops
Infinite loop	for (;;) { }	while (true) { }

6. Conclusion

This lab provided a thorough practical understanding of the for loop and nested for loops in C++. The key takeaways are:

- **for Loop:** A compact, self-contained iteration structure ideal when the number of repetitions is known before the loop begins.
- **Loop Header:** Grouping initialization, condition, and update inside the loop header keeps all control logic in one place, improving readability and reducing errors.
- **Nested for Loops:** Placing one for loop inside another enables iteration over two-dimensional structures — essential for tables, grids, and pattern printing.
- **Practical Applications:** The four lab tasks demonstrated real use cases: sequential display, formatted multiplication tables, running sum calculations, and triangle pattern generation.
- **Code Efficiency:** Using `io manip's setw()` with nested for loops produces neatly formatted output without complex string manipulation.

Mastery of the for loop is foundational to C++ programming. The concepts applied in this lab — especially nested loops and loop-controlled output — will be directly applied in upcoming labs covering arrays, functions, and algorithm design.